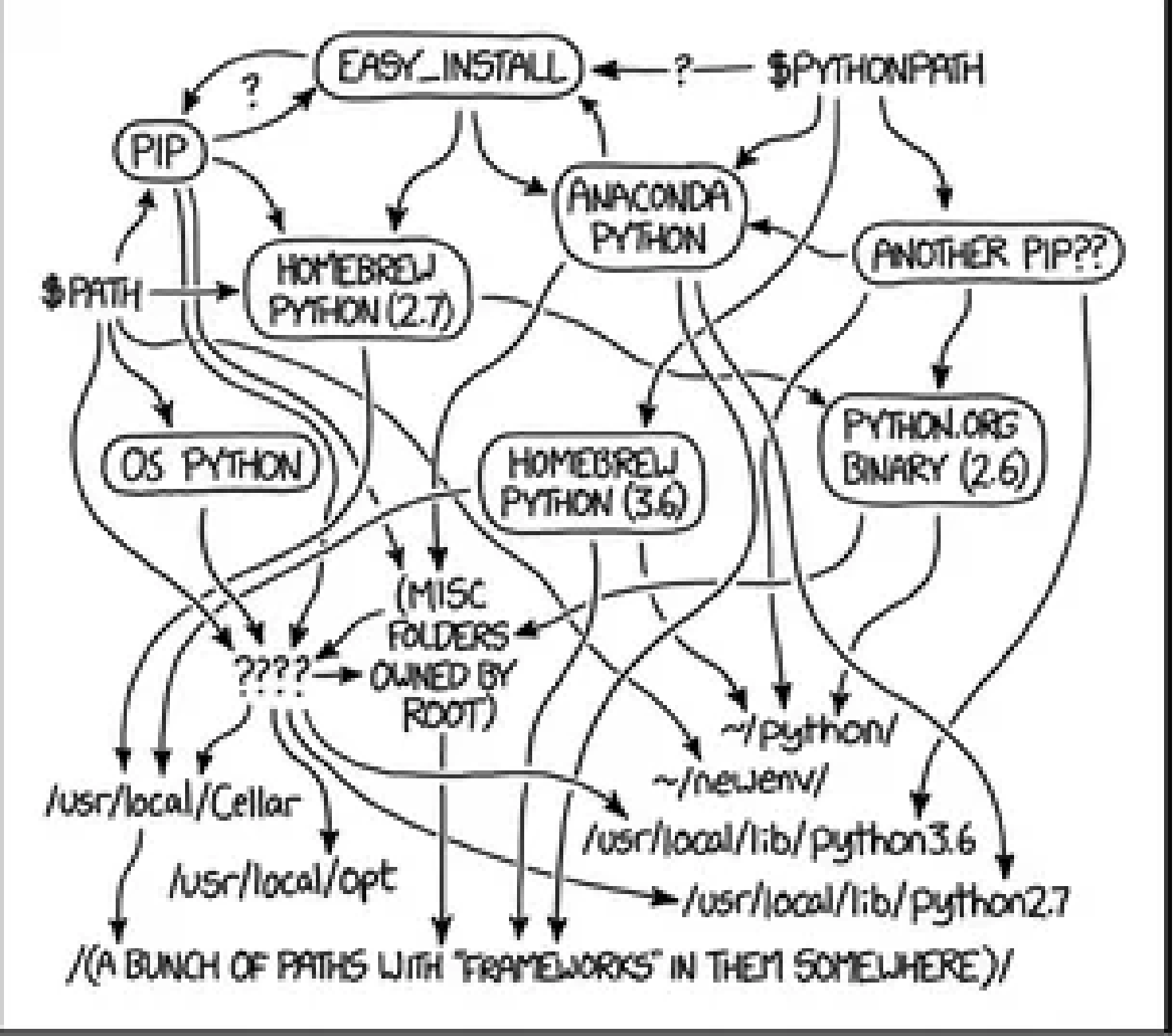


MY PYTHON ENVIRONMENT HAS BECOME SO DEGRADED THAT MY LAPTOP HAS BEEN DECLARED A SUPERFUND SITE.

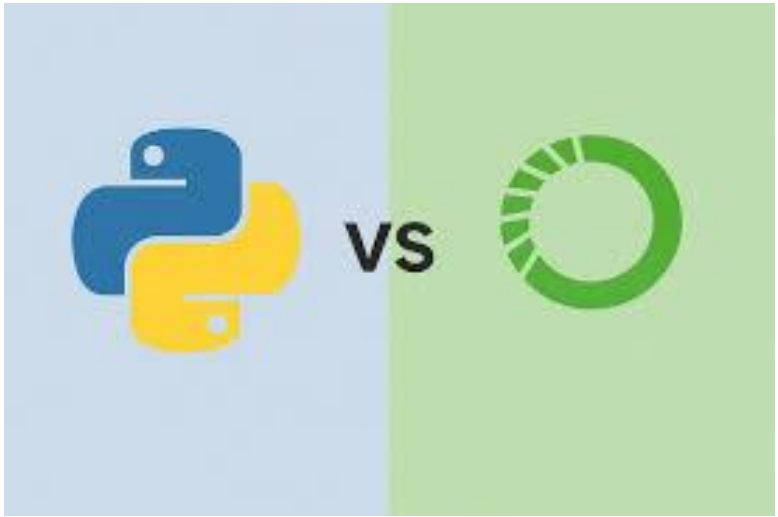
Module 1: Environment Mastery & Path Architecture

Move beyond basic script execution to understanding the underlying operating system linkages, dependency isolation, and dynamic path routing required for scalable architecture.

Presented By: Zaryab Ahmad

Virtual Environments: *venv* vs. *conda*

- The Problem: Global installations create state mutations. Upgrading a package for Project A might silently break the YOLO benchmarking script or 3D segmentation model in Project B.
- ***venv* (Standard Library):**
  - Mechanism: Creates a lightweight directory containing symlinks to the system's Python binary and a localized site-packages folder.
  - **Best for:** Standard Python microservices, web APIs, and lightweight automation.
- ***conda* (Cross-Language Package Manager):**
  - **Mechanism:** Manages not just Python packages, but underlying C/C++ libraries and binaries.
  - **Best for:** Heavy data science, computer vision, and AI workloads where system-level dependencies (like specific CUDA toolkits or OpenCV binaries) must be strictly controlled and isolated.



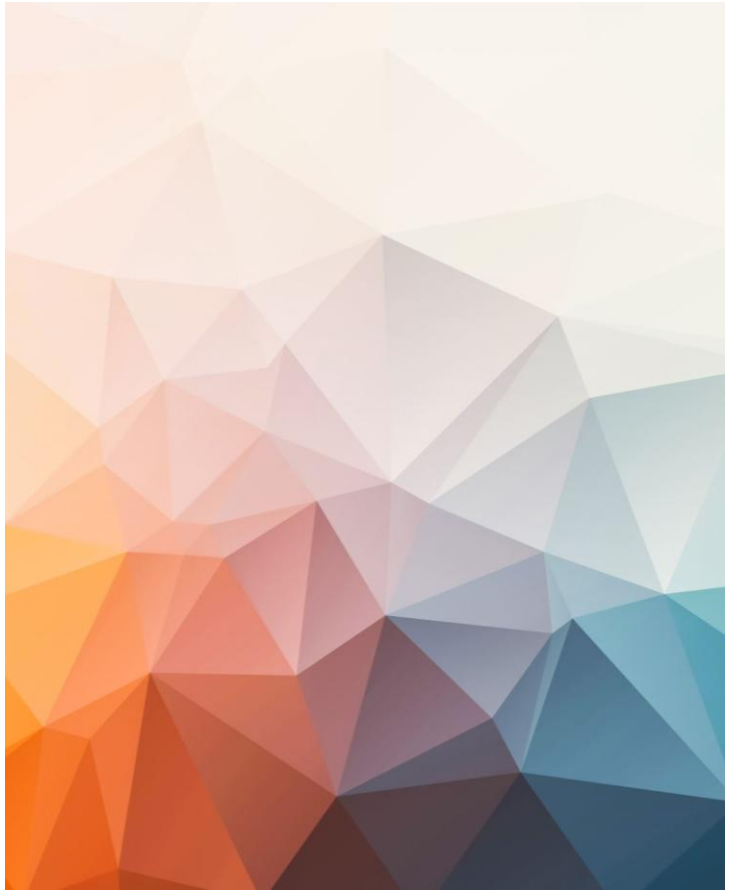
The Global $PATH$ & OS Handoff

Concept: The Operating System does not inherently know what Python is. It only knows how to search.

The $PATH Variable: An ordered list of directories the OS scans when a command is executed in the terminal.

Execution Hierarchy: If you type python, the OS executes the *first* matching binary it finds in the $PATH order.

Troubleshooting "Command Not Found": This error rarely means Python isn't installed; it means the executable's directory is missing from the system's $PATH variable.



Python's Internal Routing (sys.path)

- **Concept:** Once the OS finds Python, how does Python find your code?
- **The Resolution Order:** When you run import my_module, Python searches exactly in this order:
  - The directory containing the input script (Current Working Directory).
  - PYTHONPATH (an environment variable you can set to point to custom libraries).
  - Standard library directories (e.g., math, os).
  - site-packages (where pip or conda installs third-party tools).
- **Architectural Impact:** Understanding this order allows developers to structure repositories properly (e.g., separating src/, tests/, and data/ directories) without relying on fragile relative imports.